

Algorithm:

Step 1: Input & Initialization

- a. Read integer N (size of chessboard)
 - b. Create board[N][N] and fill with '.'
 - c. Create sets:
cols → track used columns
diag1 → track (row - col)
diag2 → track (row + col)
 - d. Create:
result → store all solutions
count = 0 → total solutions
-

Step 2: Start Function (solveNQueens)

- a. Call recursive function: solve(0)
-

Step 3: Recursive Function: solve(row)

- a. Base Case
if row == N:
store current board in result
count = count + 1
return
 - b. Try all columns
for col = 0 to N-1:
-

Step 4: Safety Check

- a. Position (row, col) is safe if:
col not in cols
(row - col) not in diag1
(row + col) not in diag2
-

Step 5: Place Queen

- a. board[row][col] = 'Q'
- b. cols.add(col)

- c. `diag1.add(row - col)`
 - d. `diag2.add(row + col)`
-

Step 6: Recurse

- a. call `solve(row + 1)`
-

Step 7: Backtrack

- a. `board[row][col] = '.'`
 - b. `cols.remove(col)`
 - c. `diag1.remove(row - col)`
 - d. `diag2.remove(row + col)`
-

Step 8: Continue Loop

Repeat for next column

Step 9: Final Output

- a. Print all solutions in result
- b. Print total count